```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns

print("All libraries loaded successfully!")
```

```
All libraries loaded successfully!
```

```
print("All libraries loaded successfully!")
```

```
All libraries loaded successfully!
```

```
from google.colab import files
uploaded = files.upload()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving cumulative.csv to cumulative.csv
```

```
df = pd.read_csv('cumulative.csv')
print("Shape:", df.shape)
print("\nColumns:", df.columns.tolist())
print("\nFirst look:")
df.head()
```

```
Shape: (9564, 50)

Columns: ['rowid', 'kepid', 'kepoi_name', 'kepler_name', 'koi_disposition', 'koi_pdispos

First look:
```

	rowid	kepid	kepoi_name	kepler_name	koi_disposition	koi_pdisposition	koi_score
0	1	10797460	K00752.01	Kepler-227 b	CONFIRMED	CANDIDATE	1.000
1	2	10797460	K00752.02	Kepler-227 c	CONFIRMED	CANDIDATE	0.969
2	3	10811496	K00753.01	NaN	FALSE POSITIVE	FALSE POSITIVE	0.000
3	4	10848459	K00754.01	NaN	FALSE POSITIVE	FALSE POSITIVE	0.000
4	5	10854555	K00755.01	Kepler-664 b	CONFIRMED	CANDIDATE	1.000

5 rows × 50 columns

```
# Keep only confirmed and false positives
df = df[df['koi_disposition'].isin(['CONFIRMED', 'FALSE POSITIVE'])]

# Create label: 1 = exoplanet, 0 = false positive
df['label'] = (df['koi_disposition'] == 'CONFIRMED').astype(int)

# Select key features
features = [
    'koi_period', 'koi_depth', 'koi_duration',
```

```
    'koi_prad', 'koi_teq', 'koi_insol',
    'koi_steff', 'koi_slogg', 'koi_srad'
]

X = df[features].fillna(df[features].median())
y = df['label']

print("Exoplanets confirmed:", y.sum())
print("False positives:", (y==0).sum())
print("Ready for training!")
```

```
Exoplanets confirmed: 2293
False positives: 5023
Ready for training!
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Scale features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train Random Forest
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)

# Results
y_pred = rf.predict(X_test)
print(classification_report(y_test, y_pred))
print("Model trained successfully!")
```

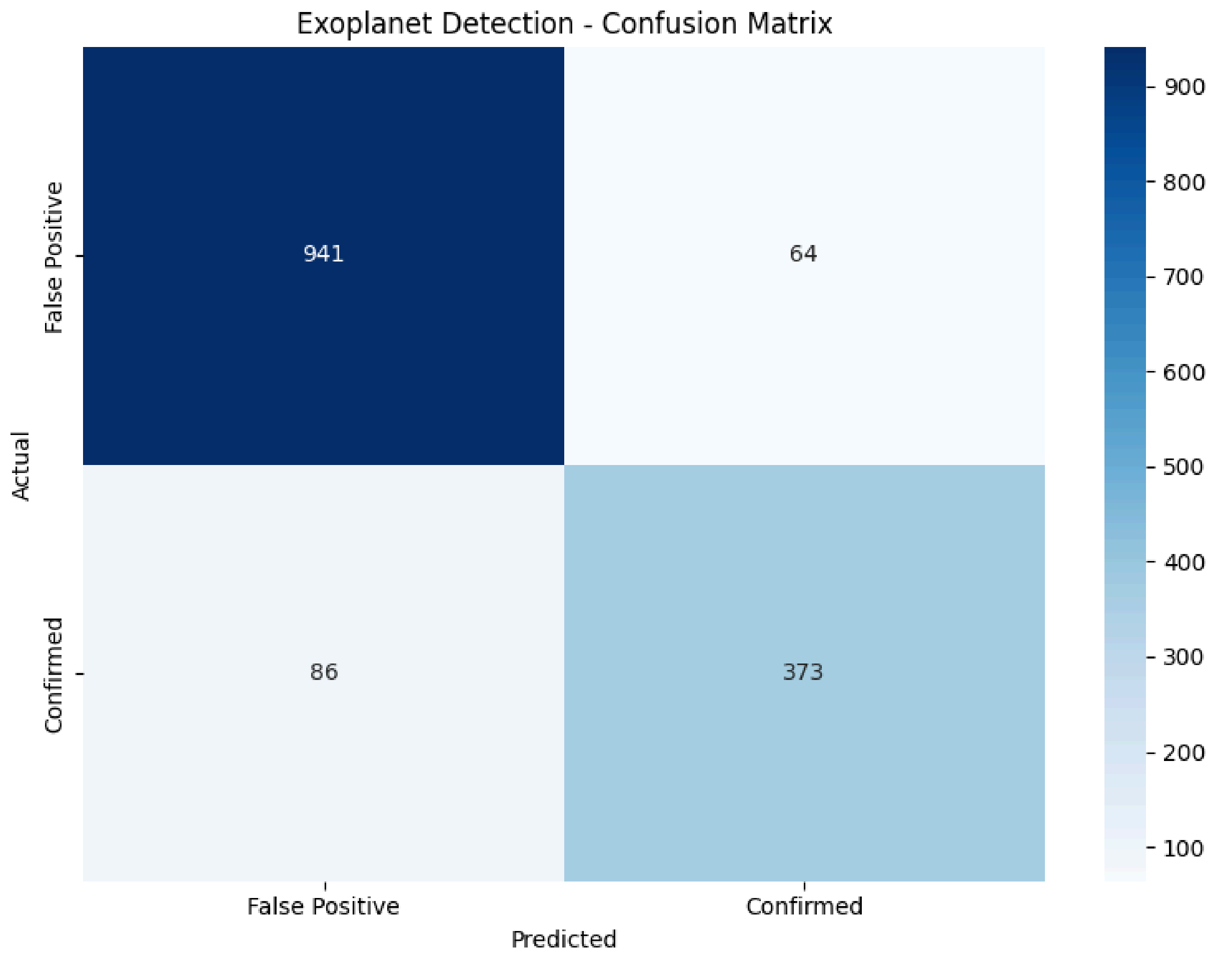
```
              precision    recall  f1-score   support

           0       0.92      0.94      0.93      1005
           1       0.85      0.81      0.83       459

    accuracy                           0.90      1464
   macro avg       0.88      0.87      0.88      1464
weighted avg       0.90      0.90      0.90      1464

Model trained successfully!
```

```
# Confusion Matrix
plt.figure(figsize=(8,6))
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['False Positive', 'Confirmed'],
            yticklabels=['False Positive', 'Confirmed'])
plt.title('Exoplanet Detection - Confusion Matrix')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.tight_layout()
plt.savefig('confusion_matrix.png')
plt.show()
print("Saved!")
```




```
Saved!
```

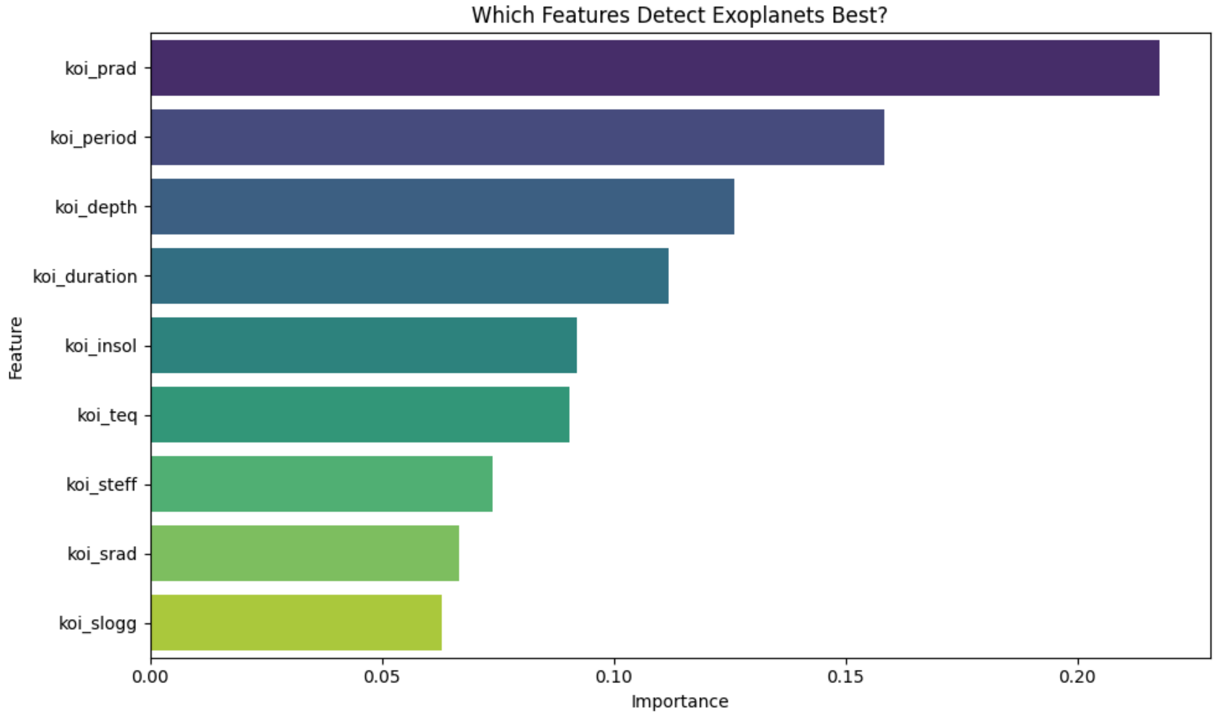
```
# Feature Importance
feat_df = pd.DataFrame({
    'Feature': features,
    'Importance': rf.feature_importances_
}).sort_values('Importance', ascending=False)

plt.figure(figsize=(10,6))
sns.barplot(x='Importance', y='Feature',
            data=feat_df, palette='viridis')
plt.title('Which Features Detect Exoplanets Best?')
plt.tight_layout()
plt.savefig('feature_importance.png')
plt.show()
print(feat_df),
```

```
/tmp/ipykernel_3648/454684754.py:8: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0.

  sns.barplot(x='Importance', y='Feature',
```




```
        Feature  Importance
3      koi_prad    0.217846
0    koi_period    0.158254
1     koi_depth    0.126040
2  koi_duration    0.111937
```